



basic education

Department:
Basic Education
REPUBLIC OF SOUTH AFRICA

SENIOR CERTIFICATE EXAMINATIONS/ NATIONAL SENIOR CERTIFICATE EXAMINATIONS

INFORMATION TECHNOLOGY P1

MAY/JUNE 2026

MARKING GUIDELINES

MARKS: 150

These marking guidelines consist of 27 pages.

GENERAL INFORMATION:

- These marking guidelines are to be used as the basis for the marking session. They were prepared for use by markers. All markers are required to attend a rigorous standardisation meeting to ensure that the guidelines are consistently interpreted and applied in the marking of candidates' work.
- Note that learners who provide an alternate correct solution to that given as example of a solution in the marking guidelines will be given full credit for the relevant solution, unless the specific instructions in the paper was not followed or the requirements of the question was not met
- **Annexures A, B, C and D** (pages 3 to 10) include the marking grid for each question.
- **Annexures E, F, G and H** (pages 11 to 27) contain examples of solutions for QUESTIONS 1 to 4 in programming code.
- Copies of **Annexures A, B, C, D and the summary for the marks of the learner** (pages 3 to 10) should be made for each learner and completed during the marking session.

ANNEXURE A**SUMMARY OF LEARNER'S MARKS:**

CENTRE NUMBER:			LEARNER'S EXAMINATION NUMBER:		
	SECTION A	SECTION B	SECTION C	SECTION D	
	QUESTION 1	QUESTION 2	QUESTION 3	QUESTION 4	GRAND TOTAL
MAX. MARKS	40	40	40	30	150
LEARNER'S MARKS					

QUESTION 1: MARKING GRID – GENERAL PROGRAMMING SKILLS

QUESTION	DESCRIPTION	MAX. MARKS	LEARNER'S MARKS
1.1	Button [1.1 - Properties] For chbQ1_1 do: • Change caption to 'Marine World' ✓ • Checked to TRUE ✓ • Font style to [fsBold] ✓ • Enabled to FALSE ✓	4	
1.2	Button [1.2 - Fudge] Extract area from cmbQ1_2 ✓ Converted to integer ✓ iNumPieces = iArea ✓ DIV 4 ✓ iPackets = iNumPieces DIV 6 ✓ iExtra = iNumPieces MOD 6 ✓ Display the output in memQ1_2 on three separate lines ✓ With correct labels ✓ With correct variables converted to string ✓	9	

1.3	Button [1.3 - Hexadecimal] Extract sHex using InputBox ✓ Display sHex on lblQ1_3Hex ✓ Initialise rDecimal and iExp to 0 ✓ Determine Length(sHex) ✓ Loop from length of sHex downto 1 ✓ Extract character ✓ Case character of ✓ '0'..'9' convert character to integer and assign to iValue variable ✓ 'A'..'F' ✓ assign corresponding integer value to iValue ✓ rDecimal = rDecimal ✓ + iValue * Power(16, iExp) ✓ increment (iExp) ✓ Display rDecimal converted to string in lblQ1_3Dec ✓	14	
1.4	Button [1.4 - Extract word] Extract sentence selected by user from lstQ1_4 ✓ Display sentence in redQ1_4 ✓ Conditional loop to: ✓ Generate random index value ✓ in the range 1 to length of sentence ✓ Until character at position of random value is space ✓ Display generated value in redQ1_4 converted to string ✓ Delete text in sentence from 1 to position of space ✓ Find the first space in the remaining part of the sentence ✓ Test if not last word in sentence ✓ Extract word from 1 to position of space - 1 ✓ Else Extract word from 1 to length of sentence - 1 ✓ Display the word in redQ1_4 ✓ Alternative for finding next space: Loop through remaining of sentence (1) Test for space (1)	13	
	TOTAL SECTION A:	40	

ANNEXURE B**QUESTION 2: MARKING GRID – DATABASE PROGRAMMING**

CENTRE NUMBER:		EXAMINATION NUMBER:	
QUESTION	DESCRIPTION	MAX. MARKS	LEARNER'S MARKS
2.1	SQL statements		
2.1.1	Button [2.1.1 - All creatures] SELECT * ✓ FROM tblCreatures ✓ ORDER BY CreatureName ✓	3	
2.1.2	Button [2.1.2 - Shallow water] SELECT HabitatID, Depth, MinTemp FROM tblHabitats ✓ WHERE Depth < 25 ✓ AND MinTemp >= 10 ✓		
2.1.3	Edit box [2.1.3 - Search habitat] SELECT HabitatName, Description ✓ FROM tblHabitats ✓ WHERE Description LIKE ✓ '%' + sFind + '%' ✓ Also ACCEPT: LIKE ' + quotedstr('%' + sFind + '%')	4	
2.1.4	Button [2.1.4 - Calculate salinity] SELECT HabitatID, SalinityConst, RIGHT(SalinityConst,2) ✓ -LEFT(SalinityConst,2) ✓ AS Salinity ✓ FROM tblHabitats ✓		
2.1.5	Button [2.1.5 - Average length] SELECT HabitatName, ROUND(AVG(CreatureLength) ✓, 2 ✓) AS AvgLength FROM tblHabitats H, tblCreatures C ✓ WHERE H.HabitatID = C.HabitatID ✓ GROUP BY HabitatName ✓ HAVING ✓ AVG(CreatureLength) > 1 ✓	7	
	Subtotal:		
		21	

QUESTION 2: MARKING GRID (CONT.)

2.2	Database Manipulation		
2.2.1	Button [2.2.1 - Update] Go to the first record in tblCreatures ✓ Loop through tblCreatures ✓ Test if sCreature ✓ is contained in tblCreatures['CreatureName'] ✓ tblCreatures.Edit ✓ tblCreatures['CreatureName'] = tblCreatures['CreatureName'] + '*' ✓ tblCreatures.Post Display the CreatureName and HabitatID ✓ in redQ2_2 inside IF-statement ✓ tblCreatures.Next ✓ End loop (tblCreatures)	9	
2.2.2	Button [2.2.2 - Habitat summary] Go the first record in tblHabitats Loop through tblHabitats ✓ Set Counter = 0 ✓ Go to the first record in tblCreatures Loop through tblCreatures ✓ If tblCreatures['HabitatID'] = tblHabitats['HabitatID'] ✓ Increment Counter ✓ tblCreatures.Next ✓ End loop (tblCreatures) If Counter > 0 ✓ Display HabitatName + #9 ✓ + Counter converted to string ✓ tblHabitats.Next ✓ End loop (tblHabitats) NOTE: One mark allocated for the correct use of Tablename.First and Tablename.Next for each table.	10	
	Subtotal:	19	
	TOTAL SECTION B:	40	

ANNEXURE C**QUESTION 3: MARKING GRID – OBJECT-ORIENTATED PROGRAMMING**

CENTRE NUMBER:		EXAMINATION NUMBER:	
QUESTION	DESCRIPTION	MAX. MARKS	LEARNER'S MARKS
3.1.1	Function generateTurtleID Function heading with string return type and result ✓ Result = UpperCase ✓ (Copy first 3 characters of fSpecies ✓ + Copy last 2 characters of fName) ✓ + fAge converted to string ✓	5	
3.1.2	Constructor Create The following must be tested <u>after</u> the provided code ✓ Test if TurtleID ✓ = 'None' ✓ fTurtleID = generateTurtleID ✓	4	
3.1.3	Procedure injuryObservation Procedure heading with string parameter ✓ Test if parameter value = 'Improving' ✓ Decrease flnjurySeverity by 2 ✓ Test If flnjurySeverity < 0 change to 1 ✓ Test if parameter value = 'Unstable' ✓ flnjurySeverity = 10 ✓	6	
3.1.4	Function getInjurySeverity Function heading with integer return type ✓ Result = flnjurySeverity ✓	2	
3.1.5	Procedure setRehabDays Procedure heading ✓ with integer parameter ✓ fRehabDays = parameter value ✓	3	
3.1.6	Function estimateReleaseDate Function heading with date parameter ✓ and date return type ✓ Result = fRehabDays + parameter value ✓	3	
	Subtotal: Object class	23	

QUESTION 3: MARKING GRID (CONT.)

QUESTION	DESCRIPTION	MAX. MARKS	LEARNER'S MARKS
3.2.1	Button [3.2.1 - Instantiate object] objSeaTurtle := ✓ TSeaTurtle.Create ✓ (sTurtleID, sName, sSpecies, iAge) ✓ Display objSeaTurtle in redQ3_2 ✓ using toString method ✓	5	
3.2.2	Button [3.2.2 - Observation] Call injuryObservation ✓ (sObservation) ✓ Test if ✓ getInjurySeverity ✓ 1 .. 4: Days = 30 ✓ 5 .. 7: Days = 60 ✓ 8 ..10:Days = 120 ✓ Call setRehabDays(Days) ✓ Call toString to display in redQ3_2 ✓	9	
3.2.3	Button [3.2.3 - Release date] Display on pnlQ3_2_3: Converted to string ✓ Call estimatedReleaseDate ✓ with dCurrentDate as argument ✓	3	
	Subtotal Form class:	17	
	TOTAL SECTION C:	40	

ANNEXURE D**QUESTION 4: MARKING GRID – PROBLEM-SOLVING PROGRAMMING**

CENTRE NUMBER:		EXAMINATION NUMBER:	
QUESTION	DESCRIPTION	MAX. MARKS	LEARNER'S MARKS
4.1.1	Button [4.1.1 - Populate arrays] AssignFile(tFile, 'Fishing.txt') ✓ Reset(tFile) ✓ Loop through the text file with correct condition ✓ Increment iNumEntries ✓ Readln(tFile, sTemp) ✓ Readln(tFile, sLine) ✓ arrSurnameName[iNumEntries] = sLine ✓ Readln(tFile, sLine) Initialise NumberFish and TotalWeight to 0 ✓ Loop through lines while sLine <> empty ✓ Increment NumberFish ✓ TotalWeight = TotalWeight + sLine as Real ✓ Readln(tFile, sLine) ✓ (initialise and change sLine) arrNumFish[iNumEntries] = NumberFish ✓ arrTotalWeight[iNumEntries] = TotalWeight ✓ CloseFile(tFile) Concepts: File Open and Reset (2) Loop through text file (1) Read a line of text file (1) Number of entries (participants) (1) Obtain (1) and save name and surname in array (1) Initialise number of fish and total weight (1) Obtain (1), calculate (1) and save the number of fish in array (1) Obtain (1), calculate (1) and save the total weight in array (1)	14	

4.1.2	Button [4.1.2 - Competition winners] Loop from 1 to iNumEntries ✓ Test each entry if number of fish is greater than 0 ✓ Calculate the average weight: arrTotalWeight[Loop] ✓ /arrNumFish[Loop] ✓ Test if average weight is equal to rHighestAvg ✓ Add arrSurnameName[Loop] to winner string ✓ Test if average is greater than rHighestAvg ✓ Replace winner string with arrSurnameName[Loop] ✓ Replace rHighestAvg with average weight ✓ End loop Display winner string in redQ4_1 ✓	10	
4.2	Button [4.2 - Pascal's triangle] Loop iRow from 3 to 7 ✓ Nested loop ✓ iCol from 2 to 14 ✓ arrPattern[iRow,iCol] = ✓ arrPattern[iRow -1,iCol -1] ✓ + arrPattern[iRow -1,iCol +1] ✓	6	
	TOTAL SECTION D: GRAND TOTAL:	30 150	

ANNEXURE E: SOLUTION FOR QUESTION 1

```
unit Question1_U;

interface

uses
  Windows, Messages, SysUtils, Variants, Classes, Graphics, Controls,
  Forms, Dialogs, ExtCtrls, StdCtrls, ComCtrls, Math, jpeg, DateUtils;

type
  TfrmQuestion1 = class(TForm)
    grpQ1_1: TGroupBox;
    grpQ1_4: TGroupBox;
    btnQ1_1: TButton;
    lstQ1_4: TListBox;
    redQ1_4: TRichEdit;
    grpQ1_3: TGroupBox;
    lblQ1_3Hex: TLabel;
    btnQ1_3: TButton;
    lblQ1_3Dec: TLabel;
    lblHex: TLabel;
    lblDec: TLabel;
    grpQ1_2: TGroupBox;
    imgFudge: TImage;
    lblArea: TLabel;
    memQ1_2: TMemo;
    btnQ1_2: TButton;
    lblSelectSentence: TLabel;
    btnQ1_4: TButton;
    cmbQ1_2: TComboBox;
    chbQ1_1: TCheckBox;
    procedure btnQ1_1Click(Sender: TObject);
    procedure btnQ1_2Click(Sender: TObject);
    procedure btnQ1_3Click(Sender: TObject);
    procedure btnQ1_4Click(Sender: TObject);
    procedure FormShow(Sender: TObject);

  private
    { Private declarations }
  public
    { Public declarations }
  end;

var
  frmQuestion1: TfrmQuestion1;

implementation

{$R *.dfm}
```

```
//=====
// Question 1.1                                     4 marks
// =====
procedure TfrmQuestion1.btnQ1_1Click(Sender: TObject);
begin
    // Question 1.1
    chbQ1_1.Caption := 'Marine World';
    chbQ1_1.Checked := True;
    chbQ1_1.Font.Style := [fsBold];
    chbQ1_1.Enabled := False;
end;
//=====
// Question 1.2                                     9 marks
// =====
procedure TfrmQuestion1.btnQ1_2Click(Sender: TObject);
var
    iArea: Integer;
    iNumPieces: Integer;
    iPackets : Integer;
    iExtra : Integer;
begin
    // Provided code
    memQ1_2.Clear;
    // Question 1.2
    iArea := StrToInt(cmbQ1_2.Text);
    iNumPieces := iArea DIV 4;
    iPackets := iNumPieces DIV 6;
    iExtra := iNumPieces MOD 6;
    memQ1_2.Lines.Add('Total pieces of fudge: ' + IntToStr(iNumPieces));
    memQ1_2.Lines.Add('Packets of fudge: ' + IntToStr(iPackets));
    memQ1_2.Lines.Add('Remaining pieces of fudge: ' + IntToStr(iExtra));
end;
//=====
// Question 1.3                                     14 marks
// =====
procedure TfrmQuestion1.btnQ1_3Click(Sender: TObject);
var
    sHex: String;
    rDecimal: Real;
    iExp, iValue, iCnt: Integer;
    cChar: Char;
begin
    // Question 1.3
    sHex := InputBox('Convert Hexadecimal to Decimal',
        'Enter hexadecimal value:', 'A2B');
    lblQ1_3Hex.Caption := sHex;
    rDecimal := 0;
    iExp := 0;
    for iCnt := Length(sHex) downTo 1 do
    begin
        cChar := sHex[iCnt];
```

```

    case cChar of
        '0' .. '9': iValue := StrToInt(cChar);
        'A': iValue := 10;
        'B': iValue := 11;
        'C': iValue := 12;
        'D': iValue := 13;
        'E': iValue := 14;
        'F': iValue := 15;
        // 'A'..'F' : iValue := ORD(cChar) - ORD('A') + 10; // Alternative
    end;
    rDecimal := rDecimal + iValue * Power(16, iExp);
    Inc(iExp);
end;
lblQ1_3Dec.Caption := FloatToStrF(rDecimal, ffFixed, 8, 0);
end;
//=====
// Question 1.4 13 marks
// =====
procedure TfrmQuestion1.btnQ1_4Click(Sender: TObject);
var
    sSentence, sWord: String;
    iPos, iLen, iRandom, iIndex: Integer;
begin
    // Provided code
    redQ1_4.Clear;
    // Question 1.4
    sSentence := lstQ1_4.Items[lstQ1_4.ItemIndex];
    redQ1_4.Lines.Add(sSentence);

    repeat
        iRandom := Random(length(sSentence) + 1) + 1;
    until sSentence[iRandom] = ' ';

    redQ1_4.Lines.Add('Generated value: ' + IntToStr(iRandom));

    Delete(sSentence, 1, iRandom);
    iPos := Pos(' ', sSentence);
    if iPos > 0 then
        sWord := Copy(sSentence, 1, iPos - 1)
    else
        sWord := Copy(sSentence, 1, length(sSentence) - 1);

    redQ1_4.Lines.Add('Extracted word: ' + sWord);
end;

// =====
// Provided code
// =====
{$REGION 'Provided code - DO NOT CHANGE'}
procedure TfrmQuestion1.FormShow(Sender: TObject);
begin
    lstQ1_4.ItemIndex := 0;
end;
{$ENDREGION}
end.

```

ANNEXURE F: SOLUTION FOR QUESTION 2

```
unit Question2_U;

interface

uses
  Windows, Messages, SysUtils, Variants,
  Classes, Graphics, Controls, Forms, Dialogs, DB, ADODB, DBConnect_U,
  Grids, DBGrids, StdCtrls, ExtCtrls, ComCtrls, Buttons;

type
  TfrmQuestion2 = class(TForm)
    dbgOne: TDBGrid;
    dbgMany: TDBGrid;
    dbgSQL: TDBGrid;
    pgcQuestion2: TPageControl;
    tsQuestion2_1: TTabSheet;
    tsQuestion2_2: TTabSheet;
    pnlHabitats: TPanel;
    pnlCreatures: TPanel;
    redQ2_2: TRichEdit;
    btnQ2_2_1: TButton;
    btnQ2_2_2: TButton;
    btnReset: TBitBtn;
    btnQ2_1_1: TButton;
    btnQ2_1_3: TButton;
    edtQ2_1_3: TEdit;
    btnQ2_1_2: TButton;
    gpb2_1_3: TGroupBox;
    btnQ2_1_4: TButton;
    btnQ2_1_5: TButton;
    procedure FormCreate(Sender: TObject);
    procedure FormClose(Sender: TObject; var Action: TCloseAction);
    procedure btnQ2_2_2Click(Sender: TObject);
    procedure btnQ2_2_1Click(Sender: TObject);
    procedure btnResetClick(Sender: TObject);
    procedure btnQ2_1_1Click(Sender: TObject);
    procedure edtQ2_1_3Change(Sender: TObject);
    procedure btnQ2_1_2Click(Sender: TObject);
    procedure btnQ2_1_4Click(Sender: TObject);
    procedure btnQ2_1_5Click(Sender: TObject);
  private
    procedure runSQL(sSQL: string);
  public
    { Public declarations }
  end;

var
  frmQuestion2: TfrmQuestion2;
  objDB: TDBConnect;
  tblCreatures, tblHabitats: TADOTable;
  sSQL : String;

implementation
{$R *.dfm}
```

```
//=====
// Question 2.1 - Section: SQL statements
//=====

//=====
// Question 2.1.1                                     3 marks
//=====
procedure TfrmQuestion2.btnQ2_1_1Click(Sender: TObject);
begin
    //Question 2.1.1

    sSQL := 'SELECT * ' +
            'FROM tblCreatures ' +
            'ORDER BY CreatureName';

    //Provided code
    runSQL(sSQL);
end;
//=====
// Question 2.1.2                                     3 marks
//=====
procedure TfrmQuestion2.btnQ2_1_2Click(Sender: TObject);
begin
    //Question 2.1.2

    sSQL := 'SELECT HabitatID, Depth, MinTemp ' +
            'FROM tblHabitats ' +
            'WHERE Depth < 25 AND MinTemp >= 10';

    //Provided code
    runSQL(sSQL);
end;
//=====
// Question 2.1.3                                     4 marks
//=====
procedure TfrmQuestion2.edtQ2_1_3Change(Sender: TObject);
    // Provided code
    //DO NOT change
var
    sFind: string;
begin
    //Question 2.1.3
    sFind := edtQ2_1_3.Text;
    //Provided code

    sSQL := 'SELECT HabitatName, Description ' +
            'FROM tblHabitats ' +
            'WHERE Description LIKE "%" + sFind + "%"';

    //Provided code
    runSQL(sSQL);
end;
```

```
//=====
// Question 2.1.4 4 marks
//=====
procedure TfrmQuestion2.btnQ2_1_4Click(Sender: TObject);
begin
    //Question 2.1.4

    sSQL := 'SELECT HabitatID, SalinityConst, RIGHT(SalinityConst,2) -
            LEFT(SalinityConst,2) AS Salinity FROM tblHabitats ';

    //Provided code
    runSQL(sSQL);
end;
//=====
// Question 2.1.5 7 marks
//=====
procedure TfrmQuestion2.btnQ2_1_5Click(Sender: TObject);
begin
    //Question 2.1.5

    sSQL := 'SELECT H.HabitatName, ROUND(AVG(CreatureLength),2)
            AS AvgLength ' +
            'FROM tblHabitats H, tblCreatures C ' +
            'WHERE H.HabitatID = C.HabitatID ' +
            'GROUP BY HabitatName ' +
            'HAVING AVG(CreatureLength) > 1';

    //Provided code
    runSQL(sSQL);
end;
```

```
//=====
// Question 2.2 – Section Delphi code
//=====

//=====
// Question 2.2.1                                     9 marks
//=====
procedure TfrmQuestion2.btnQ2_2_1Click(Sender: TObject);
var
    sCreature: String;
begin
    // Provided code
    sCreature := cmbQ2_2_1.Text;
    redQ2_2.Clear;
    redQ2_2.Lines.Add('Creature name' + #9 + 'Habitat ID');
    redQ2_2.Lines.Add('');

    // Question 2.2.1
    tblCreatures.First;
    while not tblCreatures.EOF do
    begin
        if Pos( sCreature, tblCreatures['CreatureName'] ) > 0 then
        begin
            tblCreatures.Edit;
            tblCreatures['CreatureName'] := tblCreatures['CreatureName'] + '*';
            tblCreatures.Post;
            redQ2_2.Lines.Add(tblCreatures['CreatureName'] + #9 +
                tblCreatures['HabitatID']);
        end;
        tblCreatures.Next;
    end;
end;

//=====
// Question 2.2.2                                     10 marks
//=====
procedure TfrmQuestion2.btnQ2_2_2Click(Sender: TObject);
var
    iCount: Integer;
begin
    // Provided code
    redQ2_2.Clear;
    redQ2_2.Lines.Add('Habitat name' + #9 + 'Number of creatures');
    redQ2_2.Lines.Add('');

    // Question 2.2.2
    tblHabitats.First;
    while not tblHabitats.EOF do
    begin
        iCount := 0;
        tblCreatures.First;
        while not tblCreatures.EOF do
        begin
            if tblHabitats['HabitatID'] = tblCreatures['HabitatID'] then
                Inc(iCount);
            tblCreatures.Next;
        end;
    end;
```

```
        if iCount > 0 then
            redQ2_2.Lines.Add(tblHabitats['HabitatName'] + #9 +
                               IntToStr(iCount));

            tblHabitats.Next;
        end;

    end;

end;

// =====
// Provided code
// =====
procedure TfrmQuestion2.btnResetClick(Sender: TObject);
begin
    objDB.RestoreDatabase;
    objDB.setupGrids(dbgOne, dbgMany, dbgSQL);

    redQ2_2.Clear;
end;

procedure TfrmQuestion2.FormClose(Sender: TObject; var Action:
TCloseAction);
begin
    objDB.DisconnectFromDB;
    objDB.destroy;
end;

procedure TfrmQuestion2.FormCreate(Sender: TObject);
begin
    objDB := TDBConnect.create;
    tblCreatures := objDB.getManyTbl;
    tblHabitats := objDB.getOneTbl;

    dbgOne.DataSource := objDB.getOneDS;
    dbgMany.DataSource := objDB.getManyDS;

    objDB.setupGrids(dbgOne, dbgMany, dbgSQL);

    redQ2_2.Clear;
    redQ2_2.Paragraph.Tabcount := 2;
    redQ2_2.Paragraph.Tab[0] := 150;
    redQ2_2.Paragraph.Tab[1] := 200;
end;

procedure TfrmQuestion2.runSQL(sSQL: string);
begin
    dbgSQL.DataSource := objDB.ExecuteSQL(sSQL);
    objDB.setupGrids(dbgOne, dbgMany, dbgSQL);
end;
{$ENDREGION}

end.
```

ANNEXURE G: SOLUTION FOR QUESTION 3**Object class:**

```

unit SeaTurtle_U;

interface

uses
    SysUtils, Math;

type
    TSeaTurtle = class
    private
        fTurtleID: String;
        fName: String;
        fSpecies: String;
        fAge: Integer;
        fInjurySeverity: Integer;
        fRehabDays: Integer;

    public
        Constructor Create(sTurtleID, sName, sSpecies: String; iAge: Integer);
        function generateTurtleID: String;
        function toString: String;
        procedure injuryObservation(sObservation: String);
        function getInjurySeverity: Integer;
        procedure setRehabDays(iDays: Integer);
        function estimateReleaseDate(dDate: TDate): TDate;
    end;
implementation

{ TSeaTurtle }
// =====
// Question 3.1.1 5 marks
// =====
function TSeaTurtle.generateTurtleID: String;
begin
    Result := UpperCase(Copy(fSpecies, 1, 3) + copy(fName, length(fName)-1))
                + IntToStr(fAge);
end;
// =====
// Question 3.1.2 4 marks
// =====
Constructor TSeaTurtle.Create(sTurtleID, sName, sSpecies: String;
                             iAge: Integer);
begin
    fTurtleID := sTurtleID;
    fName := sName;
    fSpecies := sSpecies;
    fAge := iAge;
    fInjurySeverity := 5;
    fRehabDays := 60;
    if fTurtleID = 'None' then
        fTurtleID := generateTurtleID;
end;

```

```
// =====
// Question 3.1.3                                     6 marks
// =====
procedure TSeaTurtle.injuryObservation(sObservation: String);
begin
    if sObservation = 'Improving' then
        begin
            fInjurySeverity := fInjurySeverity - 2;
            if fInjurySeverity < 0 then
                fInjurySeverity := 1;
        end;
    if sObservation = 'Unstable' then
        fInjurySeverity := 10;
end;
// =====
// Question 3.1.4                                     2 marks
// =====
function TSeaTurtle.getInjurySeverity: integer;
begin
    Result := fInjurySeverity;
end;
// =====
// Question 3.1.5                                     3 marks
// =====
procedure TSeaTurtle.setRehabDays(iDays: Integer);
begin
    fRehabDays := iDays;
end;
// =====
// Question 3.1.6                                     3 marks
// =====
function TSeaTurtle.estimateReleaseDate(dDate: TDate): TDate;
begin
    Result := dDate + fRehabDays;
end;
// =====
// Provided code
// =====
function TSeaTurtle.toString: String;
begin
    Result := fName + '#' + fTurtleID + #13 +
        'Species: ' + fSpecies + #13 +
        'Age: ' + IntToStr(fAge) + #13 +
        'Injury Severity: ' + IntToStr(fInjurySeverity) + #13 +
        'Rehabilitation duration: ' + IntToStr(fRehabDays);
end;

end.
```

Main Form Unit:

```
unit Question3_U;

interface

uses
  Windows, Messages, SysUtils, Variants, Classes, Graphics, Controls,
  Forms, Dialogs, ExtCtrls, StdCtrls, ComCtrls, jpeg, Spin, Populate_U,
  SeaTurtle_U, Math;

type
  TfrmQuestion3 = class(TForm)
    gpbQ3_2_1: TGroupBox;
    lblTagID: TLabel;
    lblName: TLabel;
    cmbQ3_2_1: TComboBox;
    edtQ3_2_1: TEdit;
    rgpQ3_2_1: TRadioGroup;
    lblAge: TLabel;
    spnQ3_2_1: TSpinEdit;
    btnQ3_2_1: TButton;
    imgTurtle: TImage;
    gpb3_2_2: TGroupBox;
    btnQ3_2_2: TButton;
    gpbQ3_2_3: TGroupBox;
    btnQ3_2_3: TButton;
    redQ3_2: TRichEdit;
    rgpQ3_2_2: TRadioGroup;
    pnlQ3_2_3: TPanel;
    dtpQ3_2_3Current: TDateTimePicker;
    lblCurrentDate: TLabel;
    procedure cmbQ3_2_1Change(Sender: TObject);
    procedure btnQ3_2_2Click(Sender: TObject);
    procedure btnQ3_2_3Click(Sender: TObject);
    procedure FormShow(Sender: TObject);
    procedure btnQ3_2_1Click(Sender: TObject);
  private
    { Private declarations }
  public
    { Public declarations }
  end;

var
  frmQuestion3: TfrmQuestion3;
  objSeaTurtle: TSeaTurtle;

implementation

{$R *.dfm}
```

```
// =====  
// Question 3.2.1                                     5 marks  
// =====  
procedure TfrmQuestion3.btnQ3_2_1Click(Sender: TObject);  
var  
    sOnline, sName, sSpecies, sTurtleID: String;  
    iPlace, iAge: Integer;  
begin  
    // Provided code  
    sTurtleID := cmbQ3_2_1.Text;  
    sName := edtQ3_2_1.Text;  
    sSpecies := rgpQ3_2_1.Items[rgpQ3_2_1.ItemIndex];  
    iAge := spnQ3_2_1.Value;  
    redQ3_2.Clear;  
    // Question 3.2.1  
    objSeaTurtle := TSeaTurtle.Create(sTurtleID, sName, sSpecies, iAge);  
    redQ3_2.Lines.Add(objSeaTurtle.toString);  
end;  
// =====  
// Question 3.2.2                                     9 marks  
// =====  
procedure TfrmQuestion3.btnQ3_2_2Click(Sender: TObject);  
var  
    sObservation: String;  
begin  
    // Provided code  
    sObservation := rgpQ3_2_2.Items[rgpQ3_2_2.ItemIndex];  
    // Question 3.2.2  
    objSeaTurtle.injuryObservation(sObservation);  
    case objSeaTurtle.getInjurySeverity of  
        1 .. 4: objSeaTurtle.setRehabDays(30);  
        5 .. 7: objSeaTurtle.setRehabDays(60);  
        8 .. 10: objSeaTurtle.setRehabDays(120);  
    end;  
  
    redQ3_2.Lines.Clear;  
    redQ3_2.Lines.Add(objSeaTurtle.toString)  
end;  
// =====  
// Question 3.2.3                                     3 marks  
// =====  
procedure TfrmQuestion3.btnQ3_2_3Click(Sender: TObject);  
var  
    dCurrentDate: TDate;  
begin  
    // Provided code  
    dCurrentDate := dtpQ3_2_3Current.Date;  
    // Question 3.2.3  
    pnlQ3_2_3.caption :=  
DateToStr(objSeaTurtle.estimateReleaseDate(dCurrentDate));  
end;
```

```
// =====  
// Provided code  
// =====  
{ $REGION 'Provide code' }  
procedure TfrmQuestion3.cmbQ3_2_1Change(Sender: TObject);  
begin  
    if cmbQ3_2_1.ItemIndex <> 0 then  
    begin  
        edtQ3_2_1.Text := Populate_U.arrNames[cmbQ3_2_1.ItemIndex];  
        rgpQ3_2_1.ItemIndex := Populate_U.arrSpecies[cmbQ3_2_1.ItemIndex];  
        spnQ3_2_1.Value := Populate_U.arrAges[cmbQ3_2_1.ItemIndex];  
    end  
    else  
    begin  
        edtQ3_2_1.Text := '';  
        rgpQ3_2_1.ItemIndex := -1;  
        spnQ3_2_1.Value := 0;  
    end  
end;  
  
procedure TfrmQuestion3.FormShow(Sender: TObject);  
begin  
    dtpQ3_2_3Current.Date := DATE;  
end;  
{ $ENDREGION }  
end.
```

ANNEXURE H: SOLUTION FOR QUESTION 4

```

unit Question4_U;
interface
uses
  Windows, Messages, SysUtils, Variants, Classes, Graphics, Controls, Forms,
  Dialogs, ExtCtrls, ComCtrls, jpeg, StdCtrls, Math, Grids;
type
  TfrmQuestion4 = class(TForm)
    pgcQ4: TPageControl;
    tsQ4_1: TTabSheet;
    tsQ4_2: TTabSheet;
    imgFishing: TImage;
    pnlQ4_1: TPanel;
    redQ4_1: TRichEdit;
    btnQ4_1_1: TButton;
    btnQ4_1_2: TButton;
    btnQ4_2: TButton;
    sgdQ4_2: TStringGrid;
    procedure btnQ4_2Click(Sender: TObject);
    procedure btnQ4_1_1Click(Sender: TObject);
    procedure btnQ4_1_2Click(Sender: TObject);
    procedure FormCreate(Sender: TObject);
    procedure displayInfo;
  private
    { Private declarations }
    procedure display2D;
  public
    { Public declarations }
  end;
var
  frmQuestion4: TfrmQuestion4;
  arrSurnameName: array [1 .. 30] of String;
  arrNumFish: array [1 .. 30] of Integer;
  arrTotalWeight: array [1 .. 30] of Real;
  iNumEntries: Integer = 0;

  arrPattern: array [1 .. 7, 1 .. 15] of Integer =
    ((0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0), (0, 0, 0, 0, 0, 0, 0, 1, 0, 1,
      0, 0, 0, 0, 0, 0), (0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0),
      (0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0), (0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
      0, 0, 0, 0, 0, 0), (0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0),
      (0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0));
implementation
{$R *.dfm}
{ TfrmQuestion4 }

// =====
// Question 4.1.1 14 marks
// =====
procedure TfrmQuestion4.btnQ4_1_1Click(Sender: TObject);
var
  tFile: Textfile;
  sLine, sTemp: String;
  iNumFish: Integer;
  rTotalWeight: Real;
begin
  //Question 4.1.1

```

```

AssignFile(tFile, 'Fishing.txt');
Reset(tFile);
while NOT Eof(tFile) do
begin
    Inc(iNumEntries);
    Readln(tFile, sTemp);
    Readln(tFile, arrSurnameName[iNumEntries]);
    Readln(tFile, sLine);
    iNumFish := 0;
    rTotalWeight := 0;
    while sLine <> '' do
    begin
        inc(iNumFish);
        rTotalWeight := rTotalWeight + StrToFloat(sLine);
        Readln(tFile, sLine);
    end;
    arrNumFish[iNumEntries] := iNumFish;
    arrTotalWeight[iNumEntries] := rTotalWeight;
end;
CloseFile(tFile);

//Provided code
displayInfo;
end;
// =====
// Question 4.1.2 10 marks
// =====
procedure TfrmQuestion4.btnQ4_1_2Click(Sender: TObject);
var
    rHighestAvg, rAvg: Real;
    iCnt, iNumFish: Integer;
    sOutput: String;
begin
    //Provided code
    redQ4_1.Clear;
    rHighestAvg := 0;

    //Question 4.1.2
    for iCnt := 1 to iNumEntries do
    begin
        if arrNumFish[iCnt] > 0 then
        begin
            rAvg := arrTotalWeight[iCnt] / arrNumFish[iCnt];

            if rAvg = rHighestAvg then
                sOutput := sOutput + arrSurnameName[iCnt] + #13;

            if rAvg > rHighestAvg then
            begin
                sOutput := arrSurnameName[iCnt] + #13;
                rHighestAvg := rAvg;
            end;
        end;
    end;
    redQ4_1.Lines.Add(sOutput);

    //Provided code
    redQ4_1.Lines.Add('Highest average: ' +
        FloatToStrF(rHighestAvg, ffFixed, 8, 2));
end;

```

```
// =====
// Question 4.2                                     6 marks
// =====
procedure TfrmQuestion4.btnQ4_2Click(Sender: TObject);
var
    iRow, iCol: Integer;
begin
    //Question 4.2

    for iRow := 3 to 7 do
        for iCol := 2 to 14 do
            arrPattern[iRow, iCol] := arrPattern[iRow - 1, iCol - 1] +
                                    arrPattern[iRow - 1, iCol + 1];
// Provided code
    display2D;
end;

// =====
// Provided code
// =====

{$REGION 'Provided code - Do NOT change'}

procedure TfrmQuestion4.display2D;
var
    iRow, iCol: Integer;
    iLeftPad, iRightPad: Integer;
begin
    for iRow := 0 to 6 do
        for iCol := 0 to 14 do
            if arrPattern[iRow + 1, iCol + 1] = 0 then
                sgdQ4_2.cells[iCol, iRow] := ' '
            else
                if Length(IntToStr(arrPattern[iRow+1, iCol+1])) = 1 then
                    sgdQ4_2.Cells[iCol,iRow] := ' ' +
                                                IntToStr(arrPattern[iRow+1, iCol+1])
                else
                    sgdQ4_2.Cells[iCol,iRow]:=' ' + IntToStr(arrPattern[iRow+1, iCol+1]);
end;

procedure TfrmQuestion4.displayInfo;
var
    iCnt : integer;
begin
    redQ4_1.Clear;
    redQ4_1.Paragraph.TabCount := 3;
    redQ4_1.Paragraph.Tab[0] := 50;
    redQ4_1.Paragraph.Tab[1] := 150;
    redQ4_1.Lines.Add('Surname and Name' + #9 + 'Number of fish' + #9 +
                      'Total weight');

    for iCnt := 1 to 26 do
        redQ4_1.Lines.Add(arrSurnameName[iCnt] + #9 + IntToStr(arrNumFish[iCnt])
                          + #9#9#9 + FloatToStr(arrTotalWeight[iCnt]));
end;
```

```
procedure TfrmQuestion4.FormCreate(Sender: TObject);
var
    iRow, iCol: Integer;
    iLeftPad, iRightPad: Integer;
begin
    for iRow := 0 to 1 do
        for iCol := 0 to 14 do
            if arrPattern[iRow + 1, iCol + 1] = 0 then
                sgdQ4_2.cells[iCol, iRow] := ''
            else
                if Length(IntToStr(arrPattern[iRow+1, iCol+1])) = 1 then
                    sgdQ4_2.Cells[iCol,iRow] := ' ' +
                        IntToStr(arrPattern[iRow+1, iCol+1])
                else
                    sgdQ4_2.Cells[iCol,iRow] := ' ' +
                        IntToStr(arrPattern[iRow+1, iCol+1]);
            end;
        end;
    end;

    {$ENDREGION}

end.
```